

The Blocked or Waiting state triggers (I/O requests, event waits) ,Process synchronization foundations (signals, wait queues, and communication) , Normal vs. abnormal termination system calls (e.g., exit(), abort()) ,Resource reclamation and the concepts of Zombie and Orphan processes

Code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <signal.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        /* Child process */

        printf("Child process started\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Child is waiting for 3 seconds...\n");
        sleep(3);
    }
}
```

```

        printf("Child finished waiting\n");

        printf("Child terminating normally using exit()\n");
        exit(0);
    }

    else {
        /* Parent process */

        printf("Parent process started\n");
        printf("Parent PID: %d\n", getpid());

        printf("Parent is waiting for child using wait()\n");

        wait(NULL);

        printf("Child has terminated\n");
        printf("Parent cleaned up the child process\n");
    }

    return 0;
}

```

Output:

```

geya@LenovoL480:~/OS/Practical/Week 6$ ./task1
Parent process started
Parent PID: 3529
Parent is waiting for child using wait()
Child process started
Child PID: 3530
Parent PID: 3529
Child is waiting for 3 seconds...
Child finished waiting
Child terminating normally using exit()
Child has terminated
Parent cleaned up the child process
geya@LenovoL480:~/OS/Practical/Week 6$ |

```